

Aws Assignment- Tutedude

Deploy Flask + Express on Single EC2 Instance

Architecture

User → EC2 Instance → Flask Backend + Express Frontend

Steps

1. Launch an **EC2 instance** from Amazon Web Services
2. Connect via SSH
3. Install dependencies

```
sudo apt update  
sudo apt install python3-pip nodejs npm -y
```

4. Run **Flask Backend**

```
pip install flask  
python3 app.py
```

5. Run **Express Frontend**

```
npm install  
node server.js
```

```

Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-2-208:/home/ubuntu/frontend-express# node -v
v20.20.1
10.8.2
root@ip-172-31-2-208:/home/ubuntu/frontend-express# rm -rf node_modules package-lock.json
npm install

added 76 packages, and audited 77 packages in 4s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
root@ip-172-31-2-208:/home/ubuntu/frontend-express# node server.js
Frontend running on port 3000

```

i-07e04cdf70cb51b44 (AWS_Assignment)

PublicIPs: 3.108.63.107 PrivateIPs: 172.31.2.208

```

* Support:      https://ubuntu.com/pro

System information as of Mon Mar 16 11:25:15 UTC 2026

System load:  0.08           Processes:      115
Usage of /:   36.7% of 7.57GB Users logged in:  2
Memory usage: 36%           IPv4 address for eth0: 172.31.2.208
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

73 updates can be applied immediately.
63 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

6 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Mon Mar 16 11:05:28 2026 from 13.233.177.5
ubuntu@ip-172-31-2-208:~$ sudo su
root@ip-172-31-2-208:/home/ubuntu# ls
backend-flask  backend-flask-express  frontend-express
root@ip-172-31-2-208:/home/ubuntu# rm -r backend-flask-express/
root@ip-172-31-2-208:/home/ubuntu# ls
backend-flask  frontend-express
root@ip-172-31-2-208:/home/ubuntu# cd frontend-express/
root@ip-172-31-2-208:/home/ubuntu/frontend-express# ls
node_modules  package-lock.json  package.json  server.js
root@ip-172-31-2-208:/home/ubuntu/frontend-express#

```

i-07e04cdf70cb51b44 (AWS_Assignment)

PublicIPs: 3.108.63.107 PrivateIPs: 172.31.2.208

6. Open security group ports

Service	Port
---------	------

Express	3000
---------	------

Flask	5000
-------	------

Access:

<http://EC2-PUBLIC-IP:3000>

Pretty print ☐

```
{"message": "Hello from Flask backend!"}
```

Repo structure example

project

```
|— backend-flask
|   |— app.py
|— frontend-express
|   |— server.js
|— README.md
```

Deploy Using Docker + ECR + ECS + VPC

This is the **DevOps production style architecture**.

Architecture

User

↓

Application Load Balancer

↓

ECS Cluster

↓

Docker Containers

↓

Images stored in ECR

↓

Inside VPC

Services used:

Service	Purpose
Amazon Elastic Container Registry	Store Docker images
Amazon Elastic Container Service	Run containers
Amazon Virtual Private Cloud	Networking

Step 1 – Dockerize Applications

Flask Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python","app.py"]
```

Express Dockerfile

```
FROM node:18
WORKDIR /app
COPY . .
```

RUN npm install
CMD ["node","server.js"]

Build images

docker build -t flask-backend .
docker build -t express-frontend .

Step 2 – Push Image to ECR

Login

```
aws ecr get-login-password --region ap-south-1 \  
| docker login --username AWS --password-stdin \  
<account-id>.dkr.ecr.ap-south-1.amazonaws.com
```

Tag image

```
docker tag flask-backend:latest <ECR-URI>/flask-backend:latest
```

Push

```
docker push <ECR-URI>/flask-backend:latest
```

```
Default region name [ap-south-1]:  
Default output format [json]:  
root@ip-172-31-2-208:/home/ubuntu# aws configure  
AWS Access Key ID [*****GQOG]:  
AWS Secret Access Key [None]:  
Default region name [ap-south-1]:  
Default output format [json]:  
root@ip-172-31-2-208:/home/ubuntu# rm -rf ~/.aws  
root@ip-172-31-2-208:/home/ubuntu# aws configure  
AWS Access Key ID [None]: AKIA6OZQ4BSNPYUBUL73  
AWS Secret Access Key [None]: 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  
Default region name [None]: ap-south-1  
Default output format [None]: json  
root@ip-172-31-2-208:/home/ubuntu# aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 993850559642.dkr.ecr.ap-south-1.amazonaws.com  
  
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
root@ip-172-31-2-208:/home/ubuntu# docker push 993850559642.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest  
The push refers to repository [993850559642.dkr.ecr.ap-south-1.amazonaws.com/flask-backend]  
feec13f54f58: Pushing [=====] 11.3MB/11.49MB  
46f8e5b66298: Pushed  
e77fe36c69f0: Pushed  
4e92062d58c4: Pushed  
05c88577d350: Pushing [=====] 40.98MB/53.45MB  
0b38a99061e1: Pushing [=====] 15.6MB/17.93MB  
713ad1f34d94: Pushing [==>] 36.42MB/655.7MB  
c041ff8d6c73: Pushing [=====] 51.69MB/184.9MB  
fbd6375eafc7: Waiting  
f2522c6ed78b: Waiting
```

i-07e04cdf70cb51b44 (AWS_Assignment)

PublicIPs: 3.108.63.107 PrivateIPs: 172.31.2.208

```
root@ip-172-31-2-208:/home/ubuntu# aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 993850559642.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-2-208:/home/ubuntu# docker push 993850559642.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
The push refers to repository [993850559642.dkr.ecr.ap-south-1.amazonaws.com/flask-backend]
feec13f54f58: Pushed
46f8e5b66298: Pushed
e77fe36c69f0: Pushed
4e92062d58c4: Pushed
05c88577d350: Pushed
0b38a99061e1: Pushed
713ad1f34d94: Pushed
c041ff8d6c73: Pushed
fbde375eafc7: Pushed
f2522c6ed78b: Pushed
latest: digest: sha256:0b309a43b90edab7833fa23ab897aa121aff5c21a2b60aae45cf22e9eb6b989b size: 2420
root@ip-172-31-2-208:/home/ubuntu# docker push 993850559642.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
The push refers to repository [993850559642.dkr.ecr.ap-south-1.amazonaws.com/express-frontend]
70f542db4b9a: Pushed
9ce81ccd0806: Pushed
593f042ecbef: Pushed
d2a991bcab4d: Pushed
b624aa2d5ea2: Pushed
d399c9dc306f: Pushing [=====>] 45.08MB/154MB
84f9fal79c1b: Pushed
ce84ba212e49: Pushing [=>] 17.49MB/587.7MB
e4dc8cd9ecc8: Pushing [=====>] 27.49MB/177.2MB
6428cc293366: Waiting
2f7436e79a0b: Waiting

i-07e04cdf70cb51b44 (AWS_Assignment)
PublicIPs: 3.108.63.107 PrivateIPs: 172.31.2.208
```

Step 3 – Create ECS Cluster

Create cluster in

ECS → Create Cluster → Fargate

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

Clusters (1) info

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Cap
my-cluster	0	No tasks running	0 EC2	Default	No

Last updated 17 March 2026, 00:19 (UTC+5:30)

Create cluster

Add

- Task Definition

Amazon Elastic Container Service > Task definitions

Task definition successfully created
express-task:1 has been successfully created. You can use this task definition to deploy a service or run a task. [View task definition](#)

Task definitions (2) [Info](#) Last updated 17 March 2026, 00:17 (UTC+5:30) [Deploy](#) [Create new revision](#) [Create new task definition](#)

[Filter status](#)
Active

Task definition	Status of last revision
express-task	Active
flask-task	Active

- Container
- Image from ECR

Amazon ECR > Private registry > Repositories

Private repositories (2) [View push commands](#) [Delete](#) [Actions](#) [Create repo](#)

Repository name	Repository URI copied	Created at	Tag immutability	Encryption type
express-frontend	993850559642.dkr.ecr.ap-south-1.amazonaws.com/express-frontend	16 March 2026, 20:53:24 (UTC+05.5)	Mutable	AES-256
flask-backend	993850559642.dkr.ecr.ap-south-1.amazonaws.com/flask-backend	16 March 2026, 20:53:47 (UTC+05.5)	Mutable	AES-256

Example

Image : ECR_URI/flask-backend
Port : 5000

Repeat for frontend container.

Creating service in cluster.

The screenshot shows the Amazon Elastic Container Service (ECS) console. The left sidebar contains navigation links for Express Mode, Clusters, Namespaces, Task definitions, Account settings, Amazon ECR, Repositories, AWS Batch, Online learning workshop, Documentation, Discover products, and Subscriptions. The main content area displays the 'my-cluster' overview, including the ARN, Status (Active), CloudWatch monitoring (Default), and Registered container instances (-). Below this, the 'Tasks' tab is selected, showing a table of tasks with columns for Task, Last status, Desired status, Task definition, Health status, and Created at. Two tasks are listed: 'express-task:1' (Pending) and 'flask-task:1' (Running).

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

my-cluster

Cluster overview

ARN: arn:aws:ecs:ap-south-1:993850:559642:cluster/my-cluster

Status: Active

CloudWatch monitoring: Default

Registered container instances: -

Services

Draining: -

Active: 2

Pending: 1

Running: 1

Tasks (2)

Task	Last status	Desired status	Task definition	Health status	Created at
9f2d717e43c94ddebd1a4...	Pending	Running	express-task:1	Unknown	11 seconds ago
d1900b9ee4864bd2a58e...	Running	Running	flask-task:1	Unknown	3 minutes ago

cluster->service

Configure VPC

Create or use default VPC

Allow

Port Purpose

80 HTTP

3000 Frontend

5000 Backend

project

```
├── flask-backend
│   ├── app.py
│   └── Dockerfile
├── express-frontend
│   ├── server.js
│   └── Dockerfile
└── architecture-diagram
```


└─ README.md

README should explain

2 Deployed Application URL

Example

`http://ec2-ip:3000`

or

`http://load-balancer-dns`

Implemented containerized microservice deployment using Docker, AWS ECR, ECS and VPC networking, enabling scalable cloud deployment of Flask and Node.js applications.

.